



Milestone #3

mini Database Management System Development

Reported By:

GROUP MahSQL.5

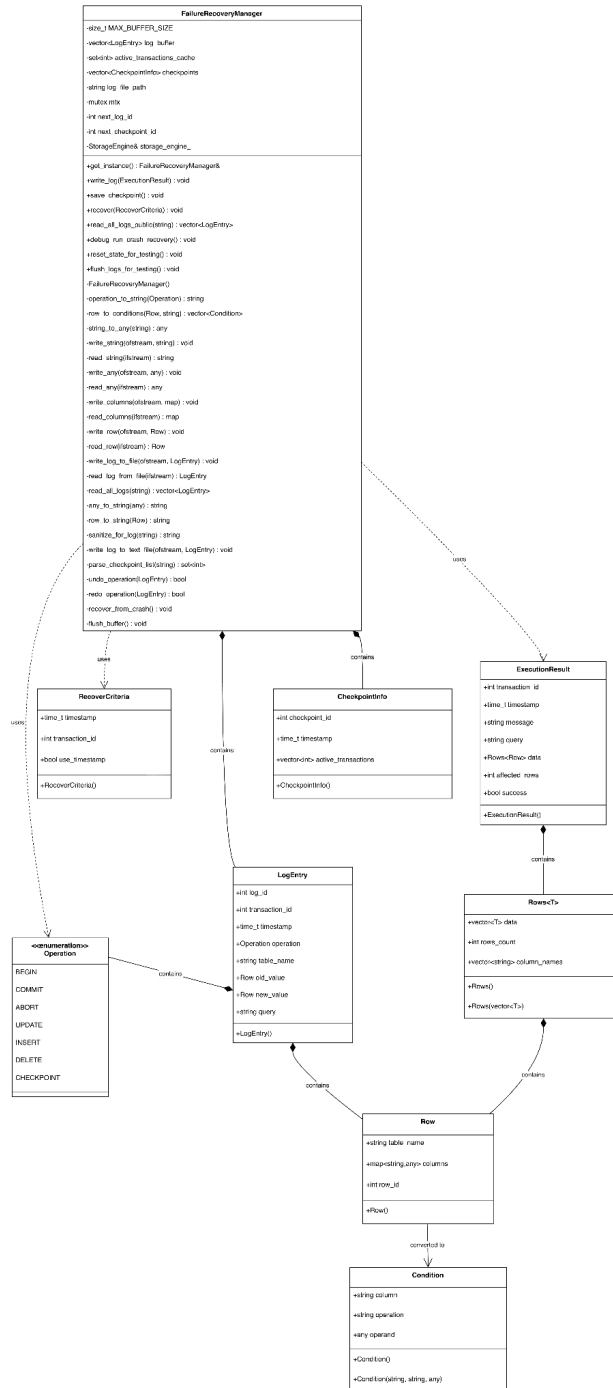
assembly

Anggota:

1. Alvin Christopher Santausa - 13523033
2. Muhammad Iqbal Haidar - 13523111
3. Ferdin Arsenarendra Purtadi - 13523117
4. Reza Ahmad Syarif - 13523119

Part I. Gambaran Umum Komponen

Berikut *class diagram* Failure Recovery Manager



Gambar *Class Diagram* Failure Recovery Manager

Kelas	Kode	Fungsi
FailureRecover yManager	<pre> class FailureRecoveryManager { public: static FailureRecoveryManager& get_instance(); FailureRecoveryManager(const FailureRecoveryManager&) = delete; FailureRecoveryManager& operator=(const FailureRecoveryManager&) = delete; ~FailureRecoveryManager(); void write_log(const ExecutionResult& info); void save_checkpoint(); void recover(const RecoverCriteria& criteria); private: FailureRecoveryManager(); const size_t MAX_BUFFER_SIZE = 50; std::vector<LogEntry> log_buffer; std::set<int> active_transactions_cache; std::vector<CheckpointInfo> checkpoints; std::string log_file_path; std::mutex mtx; int next_log_id; int next_checkpoint_id; operation_to_string(Operation op); std::vector<Condition> row_to_conditions(const Row& row, const std::string& table_name); std::any string_to_any(const std::string& str); void write_string(std::ofstream& out, const std::string& str); std::string read_string(std::ifstream& in); </pre>	Kelas ini berfungsi sebagai pusat kendali proses pencatatan log, pembuatan checkpoint, dan pemulihan. Kelas FailureRecoveryManager memiliki beberapa method yaitu: get_instance() : Mengembalikan single instance dari FailureRecoveryManager (Singleton pattern). write_log(info: ExecutionResult) : Mencatat operasi database ke log buffer. Menentukan tipe operasi (INSERT/UPDATE/DELETE/BEGIN/COMMIT/ABORT), menyimpan old_value dan new_value, lalu menambahkan ke buffer. Memanggil flush jika buffer mencapai kapasitas maksimum. save_checkpoint() : Membuat checkpoint dengan melakukan flush semua buffer ke disk, mencatat transaksi aktif, dan menyimpan checkpoint info untuk recovery point. recover(criteria: RecoverCriteria) : Melakukan recovery dengan UNDO operasi secara backward berdasarkan transaction_id. Membaca semua log dari disk, filter berdasarkan kriteria, lalu rollback operasi dari yang terbaru ke awal. operation_to_string(op: Operation) : Mengkonversi enum Operation menjadi string (BEGIN/COMMIT/ABORT/INSERT/UPDATE/DELETE/CHECKPOINT). row_to_conditions(const Row& row, const std::string&

	<pre> void write_any(std::ofstream& out, const std::any& val); std::any read_any(std::ifstream& in); void write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns); std::map<std::string, std::any> read_columns(std::ifstream& in); void write_row(std::ofstream& out, const Row& row); Row read_row(std::ifstream& in); LogEntry read_log_from_file(std::ifstream& in); void write_log_to_file(std::ofstream& out, const LogEntry& entry); std::vector<LogEntry> read_all_logs(const std::string& file_path); std::string any_to_string(const std::any& val); std::string row_to_string(const Row& row); std::string sanitize_for_log(std::string input); void write_log_to_text_file(std::ofstre am& out, const LogEntry& entry); std::set<int> parse_checkpoint_list(const std::string& query) </pre>	table_name) : Mencari identifying condition (primary key) dari sebuah row pada tabel 7. string_to_any(const std::string& str) : Mengkonversi string menjadi std::any tipe data yang diinferensi. 8. write_string(std::ofstream& out, const std::string& str) : Menulis data string dalam bentuk binary ke file. 9. read_string(std::ifstream& in) : Membaca data string dalam bentuk binary dari file. 10. write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns) : Menulis data map columns dalam bentuk binary ke file. 11. read_columns(std::ifstream& in) : Membaca data map columns dalam bentuk binary dari file. 12. write_row(std::ofstream& out, const Row& row) : Menulis data Row dalam bentuk binary ke file. 13. read_row(std::ifstream& in) : Membaca data Row dalam bentuk binary dari file. 14. read_log_from_file(std::ifstream& in) : Membaca sebuah data LogEntry dalam bentuk binary dari file. 15. write_log_to_file(std::ofstream& out, const LogEntry& entry) : Menulis sebuah data LogEntry dalam bentuk binary ke file.
--	---	---

	<pre> bool undo_operation(const LogEntry& entry); bool redo_operation(const LogEntry& entry); void recover_from_crash(); void flush_buffer(); </pre>	16. read_all_logs(const std::string& file_path) : Membaca seluruh data LogEntry dalam bentuk binary dari file. 17. any_to_string(const std::any& val) : Mengkonversi `std::any` menjadi string untuk serialisasi (mendukung tipe int, float, dan string). 18. row_to_string(const Row& row) : Melakukan serialisasi Row menjadi format string. 19. sanitize_for_log(std::string input) : Fungsi helper untuk mengganti newline menjadi spasi pada string. 20. write_log_to_text_file(std::ofstream& out, const LogEntry& entry) : Menulis sebuah data LogEntry dalam bentuk plaintext ke file. 21. flush_buffer() : Menulis semua log entries di buffer ke disk file, lalu mengosongkan buffer untuk menghemat memory. 22. undo_operation(entry: LogEntry) : Melakukan UNDO operasi berdasarkan tipe: - INSERT → DELETE row baru (new_value) - DELETE → INSERT kembali row lama (old_value) - UPDATE → Restore ke nilai lama (old_value) 23. redo_operation(entry: LogEntry) : Melakukan REDO operasi berdasarkan tipe: - INSERT → INSERT row baru (new_value) - DELETE → Hapus kembali row lama (old_value)
--	---	--

		- UPDATE → Update ke nilai baru (new_value) 24. recover_from_crash() : Melakukan recovery dari kegagalan sistem. Dimulai dengan proses mencari checkpoint terakhir, build undo_list. Kemudian, scan log setelah checkpoint untuk update status transaksi. Lalu, step REDO dan UNDO transaksi. 25. parse_checkpoint_list(const std::string& query) : Melakukan pembentukan undo_list dari log CHECKPOINT.
LogEntry	<pre> struct LogEntry { int log_id; int transaction_id; std::time_t timestamp; Operation operation; std::string table_name; Row old_value; Row new_value; std::string query; LogEntry() : log_id(-1), transaction_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Kelas ini merepresentasikan satu entri log yang disimpan saat suatu transaksi melakukan operasi.
CheckpointInfo	<pre> struct CheckpointInfo { int checkpoint_id; std::time_t timestamp; std::vector<int> active_transactions; CheckpointInfo() : checkpoint_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Merepresentasikan checkpoint yang disimpan untuk mempercepat proses recovery.
RecoveryCriteria	<pre> struct RecoverCriteria { std::time_t timestamp; int transaction_id; bool use_timestamp; }; </pre>	Kelas ini digunakan untuk menentukan bagaimana proses recovery dilakukan.

	<pre>RecoverCriteria() : timestamp(0), transaction_id(-1), use_timestamp(false) {} };</pre>	
Operation	<pre>enum class Operation { BEGIN, COMMIT, ABORT, UPDATE, INSERT, DELETE, CHECKPOINT };</pre>	Enumerasi untuk mendefinisikan jenis operasi yang dicatat dalam log
Row	<pre>struct Row { std::string table_name; std::map<std::string, std::any> columns; int row_id; Row() : row_id(-1) {} };</pre>	Untuk merepresentasikan satu baris data dalam tabel

Part II. Bagian yang Telah Selesai

Berikut adalah bagian yang berhasil dikembangkan

- Inisialisasi tipe dan struktur data yang diperlukan untuk Failure Recovery
- Pembuatan Class Diagram
- Konstruksi method write_log
- Konstruksi method save_checkpoint
- Konstruksi method recover
- Konstruksi fitur Penyimpanan Log
- Konstruksi fitur Recovery
- Implementasi System Failure Recovery dengan redo-undo

Part III. Bagian yang Belum Selesai

Berikut tugas-tugas yang masih perlu diselesaikan :

- Memastikan kembali proses integrasi dengan Storage Manager (request method `getSchema` agar bisa diakses untuk menentukan PK saat merubah row ke condition sebagai parameter yang dibutuhkan oleh method `delete_block`)
- Menambahkan handle failure recovery untuk operasi create, drop, dan alter table

Part IV. Distribusi Workload

NIM	Nama	Workload
13523033	Alvin Christopher Santausa	Pembuatan laporan, implementasi <code>recover(RecoverCriteria)</code> , integrasi storage manager (delete row) dengan failure recovery manager (undo).
13523111	Muhammad Iqbal Haidar	Pembuatan laporan, implementasi fitur penulisan log dalam bentuk binary ke file.
13523117	Ferdin Arsenarendra Purtadi	Pembuatan laporan, implementasi fungsi <code>write_log</code> , menerjemahkan <code>ExecutionResult</code> dari Query Processor menjadi <code>logEntry</code> .
13523119	Reza Ahmad Syarif	Pembuatan laporan, implementasi fitur system failure recovery

Part V. Lampiran

Sertakan lampiran yang diperlukan. Lampiran wajib adalah **tautan *GitHub Release*** untuk setiap *checkpoint*.

Github Release: [Release v3.1 · BP04/IF3140 DBMS](#)

